SISO Ontology to Computation

Jonathan Bailey

Software, languages and paradigms

Code as data

Lisp, McCarthy, 1958

Programs are written as lists, the same structure they manipulate. A program can build and run another program.

> **Contribution:** Event and Gate as one kind in a working language, and AI's first language.

> **Divergence:** assignment and mutable lists were kept for convenience, silently. The purity was available and not enforced.

Places and transitions

Petri nets, 1962

Tokens sit in places. A transition fires when all its input places hold tokens, consuming them and producing new ones.

> **Contribution:** order by causality alone. Concurrency is natural, since anything enabled can fire. A transition with several inputs is collision.

> **Divergence:** the marking, where every token currently sits, is a global state. The net is a map of positions as well as a flow.

The garden hose

Unix pipes, McIlroy memo 1964, built 1973

McIlroy asked for programs to be connected like garden hose. Each program reads a stream and writes a stream.

> **Contribution:** stream in, stream out, nearly in name.

> **Divergence:** the stream is untyped bytes, and each program is free to keep state and touch the world. The composition is pure; the parts are not.

Information packets

Flow-based programming, Morrison, IBM, late 1960s

Black-box components joined by bounded connections pass packets that are created, owned by one component at a time, and destroyed.

> **Contribution:** data with a lifetime, born and destroyed, never shared. The closest early relative of the core's Event.

> **Divergence:** components may keep state, and a packet may be modified while owned.

Actors

Hewitt, 1973

Everything is an actor. An actor receives a message and may send messages, create actors, and decide how it will handle the next message.

> **Contribution:** message passing with no shared memory, in a model built for concurrency.

> **Divergence:** the actor persists. It has an address, an identity, and behavior that changes between messages: a named thing that changes. That is everything the core's gate gave up.

Process networks

Kahn, 1974

Processes connected by queues, each reading and writing streams. The output depends only on the input streams.

> **Contribution:** determinism by construction, without global coordination.

> **Divergence:** processes keep internal state, and channels are ordered queues. Order is carried by the pipe, not by the events.

Liberation

Backus, 1977

In his Turing Award lecture, the designer of FORTRAN argued that programming was trapped in the von Neumann style: assignment statements moving one word at a time through a bottleneck.

> **Contribution:** the drift named, by one of the people who built the mainstream.

> **Divergence:** his function-level alternative was never widely adopted. Naming the drift did not stop it.

Communicating processes

Hoare, 1978

Sequential processes that interact only by synchronized message exchange.

> **Contribution:** interaction as the only form of sharing, with a mathematics to reason about it.

> **Divergence:** each process is sequential inside, and communication is a rendezvous with both parties waiting.

Share nothing

Erlang, 1986

Lightweight processes share no memory, communicate by message, and are allowed to crash. Supervisors restart them.

> **Contribution:** share-nothing design and failure as normal, proven in telephone switches.

> **Divergence:** a process holds state across messages by calling itself with new values, and has an identity others address. The actor's persistence, kept for practicality.

Enforced purity

Haskell, 1990

A language in which a function cannot have side effects unless its type says so. Input and output enter through a single marked door.

> **Contribution:** impurity made visible. The ledger, enforced by type rather than decree.

> **Divergence:** the ledger covers the language, not the runtime, which quietly updates values in place to make lazy evaluation efficient. A named purity over an unnamed concession.

Algorithmic chemistry

Fontana and Buss, early 1990s

Lambda expressions in a soup, applied to each other at random, with the products returned to the soup.

> **Contribution:** pure collision computing, with organization emerging from it.

> **Divergence:** no question, no residue, and no purpose but persistence. Early runs were taken over by expressions that copied themselves. Collision without control.

Continuous behaviors

Functional reactive programming, Elliott and Hudak, 1997

Programs as values that vary over continuous time, alongside discrete events.

> **Contribution:** time-varying flow as a first-class idea in programming.

> **Divergence:** continuous time as the foundation: the real line, the object the ontology rejects. The implementations sample discretely anyway. Engineering, again.

Map and reduce

2004

Very large datasets processed by pure functions applied in parallel, then combined.

> **Contribution:** parallel purity at industrial scale.

> **Divergence:** it is batch work and must finish before anything can be seen. Direction without a stream.

Event sourcing

Mid-2000s

Every change is stored as an event, and current state is rebuilt by replaying them.

> **Contribution:** everything is events, in production.

> **Divergence:** the log is the truth and the present is derived from history. The core's nearest neighbor, and its clearest divergence.

Serverless functions

2014

Stateless functions run when an event arrives.

> **Contribution:** event-triggered, stateless transformation at commercial scale.

> **Divergence:** the statelessness belongs to the container, not the function. State moves to a database, and side effects are the normal case. Stateless in name.

Devices

Tokens

Clay counters, Near East, from c. 8000 BCE

Small clay shapes stood for goods: a measure of grain, a sheep, a jar of oil. Later, sets of tokens were sealed inside clay envelopes with their shapes impressed on the outside. On one well-known account, writing grew out of those impressions.

> **Contribution:** the datum as a physical, immutable thing that passes between people. A sealed envelope cannot be edited, only broken.

> **Divergence:** each token stands for an object. Data begins as inventory, with the object view at its root.

The gear train

Antikythera mechanism, c. 2nd century BCE

A hand-cranked box of bronze gears. Turning the handle moves dials that show the sun, the moon's position and phase, and the 223-month cycle used to predict eclipses. A pin-and-slot pair makes the moon speed up and slow down using only fixed gears.

> **Contribution:** the knowledge is in the ratios. Cycles are encoded as integer tooth counts, as relations between wheels rather than stored values. It is the lü in bronze: discrete and relational.

> **Divergence:** the bed is frozen in bronze. Time does not run inside the mechanism. The crank supplies it from outside, the cleanest example of an outside providing direction. Nothing in it can learn.

The pinned cylinder

Automatic flute player, Banu Musa, Baghdad, 9th century

A rotating cylinder with pins opens and closes the holes of a flute. A different cylinder plays a different tune.

> **Contribution:** the program separated from the machine. Behavior becomes something that can be swapped.

> **Divergence:** the score becomes the thing kept, and performance becomes the replay of a stored sequence with its order fixed in the pins. The ontology's caution that the score is not the music applies here to the machine itself.

The combinatorial wheels

Llull, late 13th century

Concentric wheels inscribed with basic concepts. Rotating them produces every pairing, and each pairing is read as a proposition to examine.

> **Contribution:** generation by combination. New statements arise from concepts meeting, not from lookup. Collision, on paper.

> **Divergence:** the concepts are fixed and divine, and only their pairings move. These are objects colliding, not events.

Punched cards

Jacquard loom, 1804

A chain of cards controls which threads lift. The pattern travels with the cards, not the loom.

> **Contribution:** data drives the machine. The same loom weaves anything.

> **Divergence:** the chain is a complete record replayed in order. The cloth is rebuilt from history on every pass, event sourcing in silk.

Mill and store

Babbage and Lovelace, 1837–1843

The Analytical Engine separated the mill, where operations happen, from the store, where numbers wait. Lovelace saw that the engine could operate on any symbols, not only numbers, and speculated that it might compose music. She wrote that it "weaves algebraical patterns just as the Jacquard-loom weaves flowers and leaves."

> **Contribution:** transformation as a different kind of thing from data, and general symbol processing.

> **Divergence:** the store. Named places hold values that get overwritten. The first sign of drift arrives at the very beginning, as architecture.

Shared memory

Von Neumann architecture, 1945

Program and data share one memory. A single processor fetches an instruction, executes it, and fetches the next.

> **Contribution:** program as data, at the level of hardware.

> **Divergence:** nearly everything the core forbids, in one design: a shared mutable memory, a single order imposed by a counter, and a bottleneck between processor and memory that Backus later named. Almost every machine since is this one.

Dataflow machines

Dennis and others, 1970s

Hardware with no program counter. An instruction fires when its operands arrive.

> **Contribution:** order carried by data rather than imposed. This is the core's stream, in silicon.

> **Divergence:** the machines needed a matching unit to bring operands together, and matching proved costly. That is the collision concession from the Event/Gate discussion, met in hardware. Commercially, dataflow lost to von Neumann.

Ideas

Procedure texts

Old Babylonian mathematics, c. 1800 BCE

Problems were solved by worked example: take this, multiply by that, the result is the next thing. There were no symbols for unknowns and no general formulas. The method is shown in the doing. (Knuth read these texts as algorithms. The seked, annotated in the ontology, belongs beside them.)

> **Contribution:** computation as a sequence of transformations, each step consuming a value and producing the next. Shown, not said.

> **Divergence:** the order belongs to the scribe, imposed from outside and read top to bottom. The first program counter is a reader's eye.

Calculemus

Leibniz, 1666 onward

Drawing on Llull, Leibniz imagined a universal symbolic language in which disputes could be settled by calculation. He built a stepped reckoner and worked out binary arithmetic.

> **Contribution:** reasoning as computation, and binary as sufficient for it.

> **Divergence:** each symbol was meant to mirror a fixed, simple concept, a picture theory before the Tractatus. And his monads have no windows: nothing passes between them. It is the opposite of collision, from the man who made space relational.

Laws of thought

Boole, 1854

Logic as algebra over two values.

> **Contribution:** the discrete ground of every gate in every machine since.

> **Divergence:** propositions are timeless. Nothing happens in Boolean algebra. It is the logic of states, not of events.

Reduction

Lambda calculus, Church, 1936

Everything is a function, and computation is applying functions to functions and simplifying. There are no changing variables, no memory and no machine.

> **Contribution:** pure transformation as the whole of computation, with functions and data as one kind. Church and Rosser showed that if a term can be simplified to a final form, every order of simplification reaches the same one. Order does not matter, only what conditions what. This is the formal ancestor of the core's parallel property.

> **Divergence:** there is no stream and no outside. A term reduces in isolation toward a final form, a closed world with a finish line.

The tape

Turing, 1936

A head reads and writes symbols on a tape, one step at a time, according to a finite table.

> **Contribution:** every step finite and discrete. The halting problem shows that in general you cannot know whether a process will finish, which is why the core bounds retry instead of trusting it.

> **Divergence:** the tape is overwritten in place, and a single head means one thing at a time. It became the archetype of the object machine. Yet the same man later modeled how patterns form in living tissue from chemicals reacting and spreading (1952): process generating form. Turing belongs on both sides.

Logical neurons

McCulloch and Pitts, 1943

Neurons modeled as threshold units that compute logic.

> **Contribution:** the neural net begins, with many simple units and none meaningful alone.

> **Divergence:** fixed wiring and the neuron as a logic gate. Nothing learns. The bed was given, not carved.

Self-reproduction

Von Neumann, late 1940s

A machine that builds a copy of itself must use its own description twice: once as instructions to follow, and once as data to copy. This was worked out before DNA's structure was known, and DNA turned out to work the same way.

> **Contribution:** a self-extending system, and a description that is both Event and Gate.

> **Divergence:** it was realized as a grid of cells with states, the cellular automaton annotated in the ontology.

Fire together

Hebb, 1949

When one cell repeatedly helps fire another, the connection between them strengthens. This was later summarized as "cells that fire together wire together."

> **Contribution:** the flow carves the bed. Learning is use changing structure, with no separate store.

> **Divergence:** close to none. The divergence comes later, when nets trained this way are frozen.

The perceptron

Rosenblatt, 1958

A layer of adjustable weights, trained by correcting its errors.

> **Contribution:** carving made into a procedure.

> **Divergence:** the split between training and use begins to appear. The frozen riverbed starts as a convention.

The idiotypic network

Jerne, 1974

Antibodies recognize antigens and also recognize one another. The immune system becomes a network of recognition with no clean split between recognizer and recognized.

> **Contribution:** Event and Gate as one kind, in living tissue.

> **Divergence:** regulation through tolerance, deletion and suppression, distributed and never fully named. Control without a ledger.

Autopoiesis

Maturana and Varela, early 1970s

A living system is a network of processes that produces the components that produce it.

> **Contribution:** the organism as a process maintaining its pattern, the formal ground of the ontology's Modern biology entry.

> **Divergence:** operational closure. The environment perturbs but never instructs. There is no door for a question to enter and no residue to take, so it is not stream in, stream out.

Backpropagation

Rumelhart, Hinton and Williams, 1986

Error is passed backward through a layered net, and every weight moves a little way down its gradient.

> **Contribution:** carving at depth, with the seked as the learning rule. The postscript's frozen riverbed is carved this way.

> **Divergence:** nets learned this way overwrite old learning with new (catastrophic interference, McCloskey and Cohen, 1989). Freezing is the usual answer.

Replay

Hippocampal replay in sleep, Wilson and McNaughton, 1994

Patterns of activity from a rat's waking run reappear, compressed, while it sleeps.

> **Contribution:** the day carving the bed, observed directly.

> **Divergence:** none from the aim. This is a model for a shim, not a competitor.

Complementary learning systems

McClelland, McNaughton and O'Reilly, 1995

A fast learner (hippocampus) holds recent experience. A slow learner (cortex) absorbs it gradually through interleaved replay, which avoids overwriting what it already knows.

> **Contribution:** an answer to catastrophic interference: a temporary buffer whose job is to carve and then fade.

> **Divergence:** the buffer is a log of a kind. It fits the core's rule only to the extent that what consults it is the carving, not the present.

Reconstruction

Bartlett, 1932; reconsolidation, Nader and others, 2000

Remembering rebuilds rather than retrieves, and a recalled memory becomes changeable again before it settles.

> **Contribution:** the dinosaur bones, inside the head. The past is a trace read in the present and changed by the reading.

> **Divergence:** none. Placed here, out of chronological order, because it closes the thread on sleep.

Attention

The transformer, 2017

Each token's representation is built from its weighted relations to every other token in the context.

> **Contribution:** meaning by relation, the interval rather than the note. The context is the smaller river from the ontology's postscript.

> **Divergence:** the bed is frozen and illegible. Relations sit at the center, and none of it can be read off as a ledger.

Postscript

Nearly every part of the core appears somewhere above: the immutable datum in clay, order by causality in Petri nets and dataflow, pure transformation in Church, stream in and stream out in the garden hose, the ledger in Haskell's types, the merger of Event and Gate in Lisp and the immune system, the carving of the bed in Hebb and in sleep. What rarely appears is the whole arrangement, held by architecture, with each concession named. The lineage diverges where the drift section says it will: a store, an identity, an imposed order, a history consulted to learn the present.

© 2026 Jonathan Bailey. This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License.